

Final Technical Report

Riley Ashok and Presthika Vijaykumar

April 2026

1 Project Description and Scope

Our project is creating a autonomous drone show that allows the user to register drones to a fleet and then select shapes in a certain order to create the show. It will then show a visualization of the show.

The scope of this project is:

- Control multiple drones
- User defined mission plan for drones to create a custom drone show
- Drones should not collide and should have a warning for collision
- Drones should move to next shapes efficiently

2 System architecture and design choices

2.1 Basic Functionalities

Below are the basic functionalities for our drone fleet simulator, and what it must be able to do for the simulation to run:

Functionality	Description	Input	Output
Drone Registration	Adds a new drone to the fleet and initializes it in the system.	Drone ID Initial Position	Confirmation message; enables tracking and management of the drone
Mission Assignment	assigns a mission/shape formation to drones.	Drone ID(s) Mission type Target positions	Mission confirmation, allows coordinated drone tasks.
Path Planning	Calculates the optimal path for a drone.	Drone position Target position Obstacles	Waypoints/trajectory, ensures efficient and collision-free movement.
Collision Detection / Avoidance	Checks for potential collisions and adjusts paths.	Drone positions Velocity Proximity threshold	Adjusted paths/warnings/alerts, prevents collisions and ensures safety.
Simulation Update	Updates drone positions and mission progress.	Time step Drone commands	Updated positions and statuses, Advances simulation loop.
Visualization	Shows drone positions graphically, simulation data, 2D/3D, helps users understand drone behavior and mission progress.	Drone Positions Mission Plan	Graphical representation of drones, paths, and environment.

Table 1: Core functionalities of the drone fleet management system

2.2 Object Oriented Programming

Our drone fleet simulator will use four classes. One will be individual drones, one will control fleets, one will plan the mission for the fleets, and then the final one will develop simulations and graphics. A list of the classes as well as their attributes can be found below:

Class Name	Attribute / Method	Access Data Type	Description
Drone	droneID	private int	Unique ID number for drone
	position	private struct	current position of drone
	velocity	private float	current velocity of drone
	updatePosition()	public void	updates the position of drone
	detectCollision(otherDrone)	public bool	will tell if drone will collide with another drone
	addPosition()	public void	Add a new position for the drone
	getPosition()	public void	get the current position of the drone
	getPattern()	public void	get the pattern the drone is doing
	getDroneID()	public int	get the ID number of drone
FleetManager	setColor	public void	set the color of the drone LED
	drone[]	private int	array of drones in fleet
	droneAngle	private double	angle drones are flying in
	droneDistance[]	private double	array of drone distances from fleet
	dronePosition[]	private double	position of drones from fleet
	addDrone(droneID)	public void	adds a drone to the fleet
MissionPlanner	deleteDrone(droneID)	public void	deletes a drone from the fleet
	missionID	private int	Unique ID for mission element
	missionType	private string	information about mission
	planPath(droneID, target)	public vector	tell a drone where to go
	assignMission(Mission)	public void	assign a group of drones to a mission
	getMissionPlan()	public void	get the current mission of a group of drones
SimulationEngine	deleteMission(missionID)	public void	delete a mission from the queue
	addMission()	public void	add a mission to the queue
	timeStep	private float	How long mission element will take
	runStep()	public void	runs the mission element
	displayStatus	public void	displays graphics and status of fleet

Table 2: Objects for Drone Simulation Engine

2.3 System Overview

In this section, we will discuss the inputs, process, and output for the drone fleet simulator.

2.3.1 Input

1. Drone data:

- droneID
- position
- velocity

2. Mission assignments:

- shape
- target positions

3. Simulation parameters

- time step
- environment data

2.3.2 Process

1. User can create drones and fleets, setting beginning parameters
2. User enters shapes for drone show
3. MissionPlanner calculates optimal paths
4. FleetManager updates drone positions
5. Collision detection checks for other drones and potential crashes
6. Collision avoidance adjusts drone paths
7. Simulation Engine updates all statuses and generates graphics

2.3.3 Output

- Updated drone positions
- Collision avoidance actions
- Mission progress updates
- Visualization of drone show
- Errors if drones will collide or are in the same location
- Error if shape or positions are impossible for drone

A detailed diagram of the system overview can be seen below:

2.4 Functionality Tables

Functionality	Purpose	Input	Output
Drone	To create a new drone object	droneID position velocity	Confirmation message of creation of drone

Table 3: Functionalities of Drone class



Figure 1: System Process Flowchart

Functionality	Purpose	Input	Output
Fleet Manager	To create a fleet of drones to go through a show	DroneID array FleetID	Confirmation message for fleet creation

Table 4: Functionalities of Fleet Manager class

Functionality	Purpose	Input	Output
Mission Planner	To set and plan a mission for the drones to follow, likely a series of shapes	MissionID Shape Selected Drones	Confirmation of created mission

Table 5: Functionalities of Mission Planner class

Functionality	Purpose	Input	Output
SimulationEngine	To visualize the drone show and make a UI	MissionID FleetID	Simulation of the drone show thus far

Table 6: Functionalities of SimulationEngine class

3 Implementation details

3.1 Class Interactions and Responsibilities

Class Name	Responsibility	Interacts with	Reason
Drone	Create a drone object to be used in the show	DroneRegistration FleetManager MissionPlanner SimulationEngine	This is the base drone class that all other classes will use for the mission
FleetManager	Manage an array of drone and work with separate fleets of drones at a time	DroneRegistration Drone	Uses DroneRegistration to add drones to the fleet and manage an array of drone objects
MissionPlanner	Add elements to the drone show and direct fleets of drones into different shapes	FleetManager Drone	It uses fleets and drones to create the overall mission and directs the fleets into inputted positions
SimulationEngine	Display the total mission and show to the user and handle the UI	FleetManager MissionPlanner Drone	Shows the fleets and shows a visualization of the overall mission. Can also access individual drones to see their path

Table 7: Interactions between classes

3.2 Pseudocode

3.2.1 Drone Class

```

1: Class Drone
2:   Private:
3:     droneID  $\rightarrow$  integer
4:     position[3]  $\rightarrow$  array
5:     velocity[3]  $\rightarrow$  array
6:     acceleration[3]  $\rightarrow$  array
7:     color  $\rightarrow$  string
8:   Public:
9:     Function Constructor(id, x, y, z, vx, vy, vz, ax, ay, az)
10:      droneID  $\leftarrow$  id
11:      position  $\leftarrow$  [x, y, z]

```

```

12:         velocity  $\leftarrow$  [vx, vy, vz]
13:         acceleration  $\leftarrow$  [ax, ay, az]
14:     End Function

15:     Function DefaultConstructor()
16:         droneID  $\leftarrow$  0
17:         position  $\leftarrow$  [0, 0, 0]
18:         velocity  $\leftarrow$  [0, 0, 0]
19:         acceleration  $\leftarrow$  [0, 0, 0]
20:     End Function

21:     Function setDroneID(id)
22:         droneID  $\leftarrow$  id
23:     End Function
24:     Function setPosition(x, y, z)
25:         position  $\leftarrow$  [x, y, z]
26:     End Function
27:     Function setVelocity(vx, vy, vz)
28:         velocity  $\leftarrow$  [vx, vy, vz]
29:     End Function
30:     Function setAcceleration(ax, ay, az)
31:         acceleration  $\leftarrow$  [ax, ay, az]
32:     End Function
33:     Function setColor(c)
34:         color  $\leftarrow$  c
35:     End Function

36:     Function getDroneID()  $\rightarrow$  integer
37:         return droneID
38:     End Function
39:     Function getPosition()
40:         return position
41:     End Function
42:     Function getVelocity()
43:         return velocity
44:     End Function
45:     Function getAcceleration()
46:         return acceleration
47:     End Function
48:     Function getColor()
49:         return color
50:     End Function

51:     Function getSpeed()
52:         return magnitude of velocity
53:     End Function
54:     Function getVelocityAngle()

```



```

55:         return velocity angle in degrees
56:     End Function
57:     Function getDistanceFromOrigin()
58:         return distance from origin
59:     End Function
60:     Function getDistanceFromOther(otherDrone)
61:         Calculate dx, dy, dz
62:         return distance
63:     End Function
64:     Function getAccelerationMagnitude()
65:         return magnitude of acceleration
66:     End Function

67:     Function displayInfo()
68:         Output droneID
69:         Output position
70:         Output velocity
71:         Output acceleration
72:         Output color
73:     End Function

74:     Function updatePosition(time)
75:         position  $\leftarrow$  position + velocity  $\times$  time
76:         Output updated position
77:     End Function
78:     Function updateVelocity(ax, ay, az, time)
79:         velocity  $\leftarrow$  velocity + acceleration  $\times$  time
80:         Output updated velocity
81:     End Function
82:     Function applyAcceleration(ax, ay, az, time)
83:         Call updateVelocity(ax, ay, az, time)
84:         Call updatePosition(time)
85:     End Function

86:     Function stop()
87:         velocity  $\leftarrow$  [0, 0, 0]
88:         Output "Drone stopped"
89:     End Function
90:     Function reset()
91:         position  $\leftarrow$  [0, 0, 0]
92:         velocity  $\leftarrow$  [0, 0, 0]
93:         acceleration  $\leftarrow$  [0, 0, 0]
94:         Output "Drone reset"
95:     End Function

96:     Function detectCollision(otherDrone, collisionDistance)
97:         Calculate distance

```

```

98:
99: if distance < collisionDistance then
100:     Output "Collision detected"
101:     return 1
102:
103: else
104:     Output "No collision"
105:     return 0
106:
107: end if
108:     End Function
109:     Function avoidCollision(otherDrone, collisionDistance)
110:
111:     if detectCollision(otherDrone, collisionDistance) == 1 then
112:         velocity  $\leftarrow$  negative of velocity
113:         Output updated velocity
114:
115:     end if
116:     End Function
117: End Class

```

3.2.2 FleetManager Class

```

1: Class FleetManager
2:     Private:
3:         drone[]  $\rightarrow$  array of Drone objects
4:         droneCount  $\rightarrow$  integer
5:     Public:
6:         Function AddDrone(droneID)
7:
8:         for each drone in drone[] do
9:
10:             if drone.ID == droneID then
11:                 Output "Drone with ID [droneID] already exists"
12:                 return
13:
14:             end if
15:
16:         end for
17:         drone[droneCount].ID  $\leftarrow$  droneID
18:         droneCount  $\leftarrow$  droneCount + 1
19:         Output "Drone with ID [droneID] added"
20:     End Function
21:
22:     Function DeleteDrone(droneID)

```

```

23: for each drone in drone[] do
24:
25:   if drone.ID == droneID then
26:     Shift all drones after current index left by one
27:     droneCount  $\leftarrow$  droneCount - 1
28:     Output "Drone with ID [droneID] deleted"
29:     return
30:
31:   end if
32:
33: end for
34:   Output "Drone with ID [droneID] not found"
35:   End Function
36: End Class

```

3.2.3 MissionPlanner Class

```

1: Class MissionPlanner
2:   Private:
3:     missionID  $\rightarrow$  integer
4:     missionName  $\rightarrow$  string
5:     missionType  $\rightarrow$  string
6:     missionRunNames[10]  $\rightarrow$  array
7:     missionRunCount  $\rightarrow$  integer
8:   Public:
9:     Function Constructor(ID, name, type)
10:       missionID  $\leftarrow$  ID
11:       missionName  $\leftarrow$  name
12:       missionType  $\leftarrow$  type
13:       missionRunCount  $\leftarrow$  0
14:     End Function
15:
16:     Function DefaultConstructor()
17:       missionID  $\leftarrow$  0
18:       missionName  $\leftarrow$  "Default Mission"
19:       missionType  $\leftarrow$  "Default Type"
20:       missionRunCount  $\leftarrow$  0
21:     End Function
22:
23:     Function setMissionID(ID)
24:       missionID  $\leftarrow$  ID
25:     End Function
26:
27:     Function setMissionName(name)
28:       missionName  $\leftarrow$  name
29:     End Function
30:
31:     Function setMissionType(type)
32:       missionType  $\leftarrow$  type
33:     End Function

```

```

28:         missionType ← type
29:     End Function

30:     Function getMissionID() → integer
31:         return missionID
32:     End Function
33:     Function getMissionName() → string
34:         return missionName
35:     End Function
36:     Function getMissionType() → string
37:         return missionType
38:     End Function

39:     Function displayMissionInfo()
40:         Output "Mission Info"
41:         Output missionID
42:         Output missionName
43:         Output missionType
44:     End Function

45:     Function assignMission(fleet)
46:         Assign mission to fleet using fleet ID
47:     End Function
48:     Function assignMission(drone)
49:         Assign mission to drone
50:     End Function

51:     Function deleteMission()
52:         missionID ← 0
53:         missionName ← "Deleted Mission"
54:         missionType ← "Deleted Type"
55:     End Function

56:     Function addMissionElement(name)
57:
58:     if missionRunCount < 10 then
59:         missionRunNames[missionRunCount] ← name
60:         missionRunCount ← missionRunCount + 1
61:
62:     else
63:         Output "Cannot add more elements"
64:
65:     end if
66:     End Function
67: End Class

```

3.2.4 SimulationsEngine class

```
1: Class SimulationEngine
2:   Private:
3:     timeStep  $\rightarrow$  float
4:     totalSteps  $\rightarrow$  integer
5:     currentStep  $\rightarrow$  integer
6:     fleet  $\rightarrow$  FleetManager
7:   Public:
8:     Function Constructor(timeStep, totalSteps, fleet)
9:
10:  if timeStep  $\leq$  0 then
11:    Output “Invalid time step”
12:    return
13:
14:  end if
15:    this.timeStep  $\leftarrow$  timeStep
16:    this.totalSteps  $\leftarrow$  totalSteps
17:    this.currentStep  $\leftarrow$  0
18:  End Function
19:
20:  Function RunStep()
21:  if timeStep  $\leq$  0 then
22:    Output “Invalid time step. Must be greater than 0”
23:    return
24:
25:  end if
26:    currentStep  $\leftarrow$  currentStep + 1
27:    Output “— Step [currentStep] of [totalSteps] —”
28:     $\triangleright$  Update all drone positions
29:
30:  for each drone i in fleet do
31:    Call drone[i].updatePosition(timeStep)
32:
33:  end for
34:     $\triangleright$  Check for collisions
35:
36:  for each drone i in fleet do
37:
38:    for each drone j in fleet where j > i do
39:
40:      if drone[i].detectCollision(drone[j]) then
41:        Output “Warning: Collision detected between
42: Drone [i] and Drone [j]”
```

```

43:         end if
44:
45:     end for
46:
47: end for
48:     Call DisplayStatus()
49:     End Function

50:     Function RunFullSimulation()
51:
52: if totalSteps  $\leq$  0 then
53:     Output "Error: No simulation steps set"
54:     return
55:
56: end if
57:     Output "Starting simulation..."
58:
59: while currentStep < totalSteps do
60:     Call RunStep()
61:
62: end while
63:     Output "Simulation complete"
64:     End Function

65:     Function DisplayStatus()
66:     Output "Current Step: [currentStep] / [totalSteps]"
67:     Output "Drones in fleet: [droneCount]"
68:
69: for each drone  $i$  in fleet do
70:     Output "Drone [ID] - Position: [position]"
71:
72: end for
73:     End Function
74: End Class

```

4 Experimental results or evaluation

5 Limitations

6 Conclusions and Proposed Future Work

INSERT CONCLUSION HERE